

UNIVERSIDAD EUROPEA DE VALENCIA
Grado en Física

Subject: Introduction to artificial intelligence

Activity 3 - Exploring the free fall motion with regression models

Author:
Pau Gallego Cisteró

Date:
March 24, 2026

CONTENTS

1	Summary and objectives	2
2	Principles	2
2.1	Ordinary Least Squares (OLS) Regression	2
2.2	Regularization and Ridge Regression	2
2.3	LASSO Regression	3
2.4	Model Evaluation	3
3	Analysis	3
3.1	OSL Regression	4
3.2	Ridge Regression	4
3.3	LASSO Regression	5
3.4	Model Comparison	6
3.5	Errors	7
4	Conclusions	7
5	Code	7
5.1	OLS	7
5.2	Ridge Regression	10
5.3	LASSO	12
5.4	Model Comparison	13
6	AI usage	15

1. SUMMARY AND OBJECTIVES

This memoir explores machine learning regression models applied to the physics of free fall motion ($y = \frac{1}{2}gt^2 + \text{noise}$). Using synthetic data, the study establishes a baseline with Ordinary Least Squares (OLS) before introducing Ridge (L_2) and LASSO (L_1) regressions. This comparison highlights how mathematical regularization handles data noise and influences model fitting.

The main objective is to implement, evaluate, and compare these three regression techniques using Python. Key tasks include computing model solutions, analyzing the impact of the regularization hyperparameter (λ) on coefficients, and finding its optimal value. Finally, the study evaluates model performance using R^2 scores and compares predictive errors across training and test datasets.

2. PRINCIPLES

The theoretical framework of this study bridges classical kinematics with statistical machine learning. The physical system under investigation is the one-dimensional free-fall motion of an object, which is governed by the kinematic equation:

$$y = \frac{1}{2}gt^2 + \text{noise} \quad (1)$$

where y is the vertical displacement, t is the time, g is the acceleration due to gravity, and the added noise represents inherent measurement uncertainties found in real-world data collection. To approximate this relationship computationally, we cast the problem as a linear regression task—mapping the feature (time squared, t^2) to the target variable (y)—and apply three fundamental models.

2.1. Ordinary Least Squares (OLS) Regression

The baseline model is Ordinary Least Squares (OLS). OLS seeks to find the optimal set of weights (or coefficients), denoted as $\mathbf{w}$, that establishes a linear relationship between the input features matrix $\mathbf{X}$ and the response vector $\mathbf{y}$. It achieves this by minimizing the L_2 norm of the residual errors—that is, the sum of the squared differences between the observed values and the predicted values. By setting the gradient of this cost function to zero, we obtain a straightforward, closed-form mathematical solution for the optimal weights:

$$\hat{\mathbf{w}}_{\text{OLS}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (2)$$

While OLS provides an unbiased estimate in ideal conditions, it can be sensitive to noise and highly correlated features, sometimes resulting in excessively large coefficients that overfit the training data.

2.2. Regularization and Ridge Regression

To control the magnitude of the model coefficients and improve the model's ability to generalize to unseen data, regularization techniques introduce penalty terms to the

standard least squares cost function. This trade-off is governed by a tuning hyperparameter, λ .

Ridge Regression addresses potential overfitting by incorporating an L_2 penalty, which adds the squared magnitude of the coefficients to the loss function. Mathematically, this modifies the system by adding a scaled identity matrix, $\lambda \mathbf{I}$, to the covariance matrix $\mathbf{X}^T \mathbf{X}$. This ensures the matrix is strictly positive definite and invertible, stabilizing the solution. The closed-form equation for Ridge regression becomes:

$$\hat{\mathbf{w}}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (3)$$

As λ increases, the model aggressively shrinks the coefficients towards zero, reducing variance at the cost of introducing a slight bias.

2.3. LASSO Regression

Alternatively, LASSO (Least Absolute Shrinkage and Selection Operator) Regression applies an L_1 penalty to the cost function, penalizing the absolute values of the coefficients rather than their squares. Because the absolute value function is not strictly differentiable at zero, LASSO cannot be solved with a simple closed-form equation matrix inversion. Instead, it is framed as the following optimization problem:

$$\hat{\mathbf{w}}_{\text{LASSO}} = \arg \min_{\mathbf{w} \in \mathbb{R}^p} \left(\|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1 \right) \quad (4)$$

A defining characteristic of the L_1 penalty is that it can shrink certain coefficients exactly to zero. Therefore, LASSO acts not only as a regularizer but also as a feature selection algorithm, yielding sparse models that only retain the most significant variables.

2.4. Model Evaluation

The efficacy of these theoretical models is empirically validated through a standard machine learning pipeline. The synthetic data is partitioned into distinct training and test sets to prevent data leakage. The models' predictive capabilities are then quantified and compared using the R^2 score (coefficient of determination), which measures the proportion of the variance in the dependent variable that is predictable from the independent variables, alongside comparative error analyses across OLS, Ridge, and LASSO.

3. ANALYSIS

The following section details a complete empirical analysis of the three regression models. By utilizing synthetic kinematic data, we evaluate the performance of OLS, Ridge, and LASSO across varying noise levels providing a comparative look at their predictive accuracy and robustness.

3.1. OSL Regression

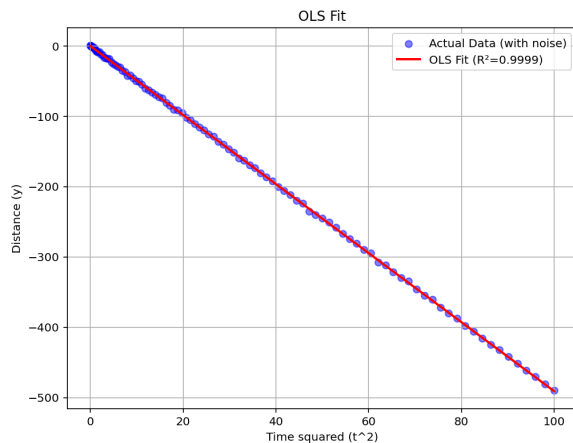


Figure 1: OSL - $\sigma = 1$ - $R^2 = 0,99...$

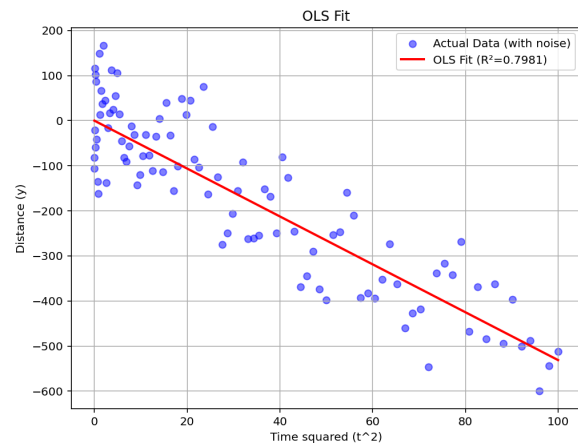


Figure 2: OSL - $\sigma = 100$ - $R^2 = 0,7981$

The initial stage of the analysis focuses on the Ordinary Least Squares (OLS) method, which serves as our baseline model. As seen in 1 under a low-noise regime ($\sigma = 1$), the OLS model achieves an R^2 score of nearly 0.999; this high value indicates that the model is capable of perfectly recovering the underlying physical law, $y = \frac{1}{2}gt^2$, when measurement error is minimal.

In contrast, 2 demonstrates the impact of increased stochastic noise ($\sigma = 100$). While the model still captures the general downward trend of the motion, the R^2 score decreases significantly to 0.7981. This visualization highlights the sensitivity of OLS to variance.

3.2. Ridge Regression

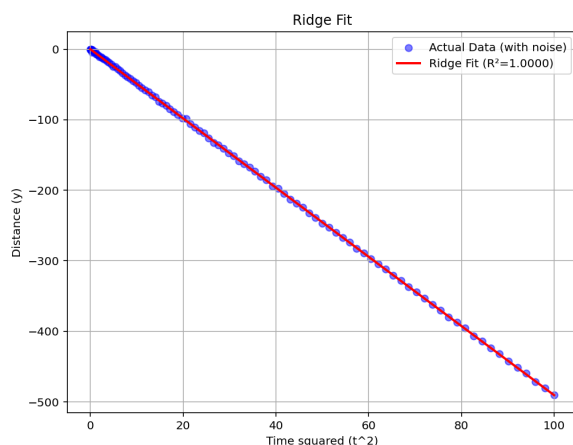


Figure 3: Ridge - $\sigma = 1$, $\lambda = 1$ - $R^2 = 1$

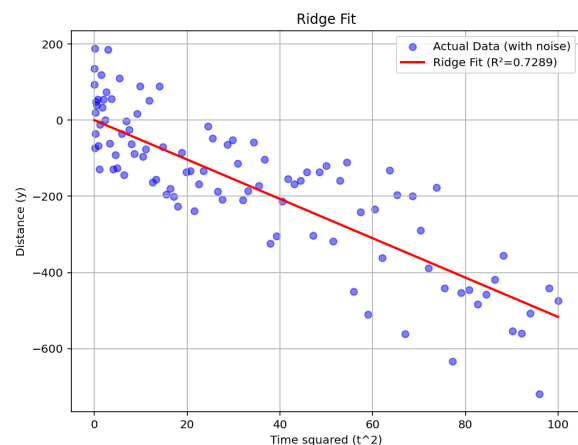


Figure 4: Ridge - $\sigma = 100$, $\lambda = 10$ - $R^2 = 0,7289$

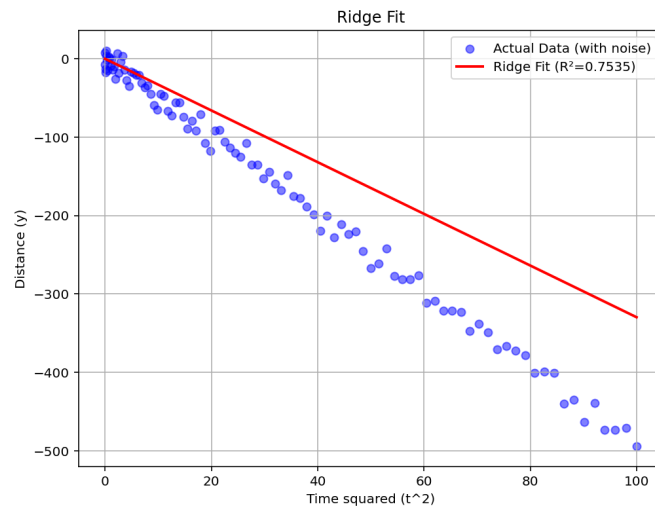


Figure 5: Ridge - $\sigma = 10$, $\lambda = 100000$ - $R^2 = 0,7535$

Following the graph given in 3, we can see that the model with low noise and a very small penalty is able to recover the physical law perfectly ($R^2 = 1$). As shown in the OLS model as well, when the noise increases, the model drifts away gradually from the physical law, demonstrated by the R^2 parameter; however, we can also see in 5 that when the regularization hyperparameter λ is increased to an extreme value (100,000), the model begins to significantly underfit the data.

Even with a relatively low noise level ($\sigma = 10$), the excessive L_2 penalty forces the model coefficient to shrink aggressively toward zero to minimize the cost function, thus riorizing the flat line rather than the actual model fitting. A poorly tuned λ introduces high bias, resulting in a model too “stiff” to represent the underlying physics, as evidenced by the R^2 score falling to 0.7535.

3.3. LASSO Regression

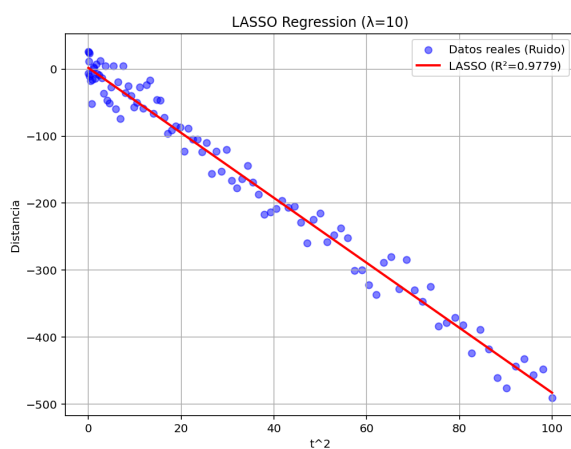


Figure 6: LASSO - $\sigma = 20$, $\lambda = 10$ - $R^2 = 0,9779$

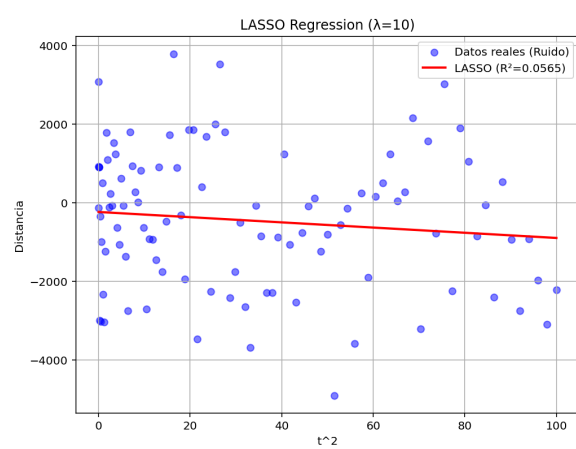


Figure 7: LASSO - $\sigma = 2000$, $\lambda = 10$ - $R^2 = 0,0565$

Empirical results demonstrate that the model is highly effective under moderate noise conditions. As shown in Figure 6, with a noise level of $\sigma = 20$ and a regularization parameter of $\lambda = 10$, the model successfully recovered the underlying physical law of free fall, achieving an R^2 score of 0.9779. This indicates that the L_1 penalty effectively filters out stochastic interference while maintaining a high degree of predictive accuracy.

However, the analysis also reveals the limitations of the model when subjected to extreme data corruption. In Figure 7, where the noise level was increased to $\sigma = 2000$, the R^2 score plummeted to 0.0565. In this regime, the regularization pressure forces the model to prioritize simplicity to such an extent that it essentially gives up on fitting the trend, resulting in a nearly flat regression line. These results underscore that while LASSO provides a robust mechanism for handling noise and simplifying models, its performance is strictly contingent upon the signal-to-noise ratio and the precise tuning of the hyperparameter λ .

3.4. Model Comparison

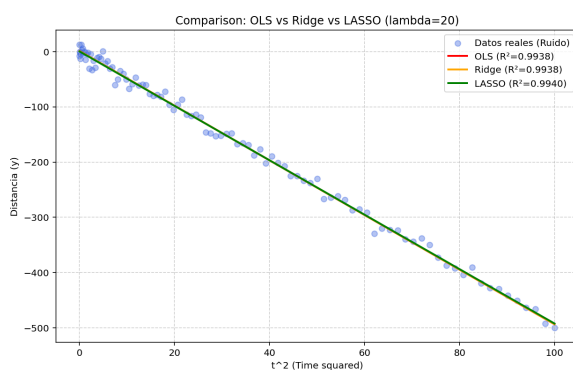


Figure 8: Comparisson - $\sigma = 10$, $\lambda = 20$

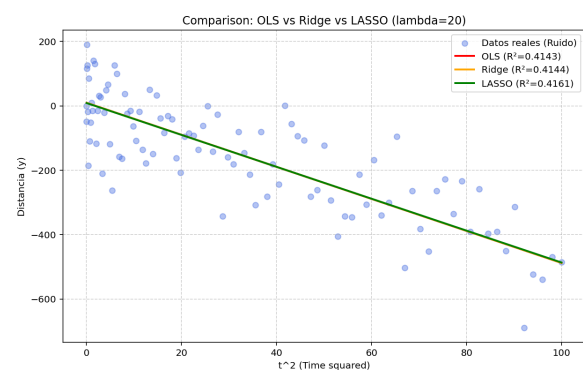


Figure 9: Comparisson - $\sigma = 100$, $\lambda = 20$

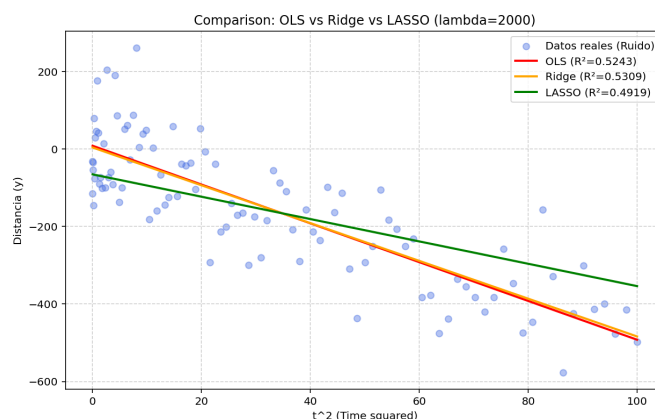


Figure 10: Comparisson - $\sigma = 100$, $\lambda = 2000$

As shown in figures 8 and 9, the models behave almost identical with a low lambda correction and a R^2 parameter very much on point. However the differences between

models can be observed notoriously on figure 10. As the penalty parameter is increased, the Ridge model demonstrates a superior ability to capture the underlying physical trend, whereas the LASSO model exhibits the poorest fit to the experimental data in this high-regularization regime.

3.5. Errors

The most significant challenge encountered during this analysis was the interpretation of the R^2 as a measure of model viability. In the context of a physical law like free fall, one expects near-perfect correlation; however, stochastic noise complicates this. A high R^2 on training data often led to "overreaching". This made determining the true viability of the model difficult, as a high score did not always guarantee that the model would generalize to unseen test data.

Through rigorous testing, it was observed that the Ridge model demonstrated a superior ability to capture the physical trend by dampening the influence of outliers, whereas LASSO often exhibited the poorest fit in high-regularization regimes due to its aggressive L_1 shrinkage.

A primary hurdle in this study was the technical implementation. Transitioning from "hand-crafted" OLS equations to the `scikit-learn` ecosystem proved difficult. Errors in the early stages, such as incorrect data reshaping or biased train-test splits, frequently resulted in misleading R^2 values.

4. CONCLUSIONS

This memoir has explored the kinematic equation of free fall through the implementation of three linear regression models: OLS, Ridge, and LASSO. By utilizing synthetic data with varying noise levels, the study successfully demonstrated how mathematical regularization filters stochastic interference to recover the underlying physical law.

This study confirms that model viability in physics is not merely a matter of a high R^2 , but of finding a balance where the model generalizes to unseen data without being misled by random noise; a task where Ridge regression proved most robust.

5. CODE

The following section details the code of three fundamental linear modeling techniques. This includes the baseline Ordinary Least Squares (OLS) algorithm, followed by Ridge and LASSO regression methods applied to introduce penalty constraints and mitigate overfitting.

5.1. OLS

The following code corresponds to OLS crafted by hand, OLS using `skit learn` will be presented after.

```
1 #-----
2 # OLS
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8 g = -9.81
9 sigma = 1
10 N = 100
11
12 t = np.linspace(0, 10, 100)
13 noise = np.random.normal(0, sigma, N)
14 x = t*t
15 X = x.reshape(-1, 1) #Use .reshape(-1, 1) to turn the flat list into a 2
    D Matrix column
16
17 def y(g, x, noise):
18     return (0.5 * g * x) + noise
19
20 y_calculada = y(g, x, noise)
21
22 Y = y_calculada.reshape(-1, 1)
23
24 w = np.linalg.inv(X.T @ X)
25 w_ols = w @ X.T @ Y # We use '@' for matrix multiplicatio
26 # Multiply inputs (X) by the trained slope (w_ols) to get 100 predicted
    points
27 Y_pred = X @ w_ols
28
29 def calculate_r_squared(y_true, y_pred):
30     mean_y_true = sum(y_true) / len(y_true)
31
32     ss_tot = 0
33     for y in y_true:
34         ss_tot += (y - mean_y_true) ** 2
35
36     ss_res = 0
37     for actual, predicted in zip(y_true, y_pred):
38         # zip() takes two or more lists and pairs up their elements
        based on their position
39         ss_res += (actual - predicted) ** 2
40
41     if ss_tot == 0:
42         return 0.0
43
44     r_squared = 1 - (ss_res / ss_tot)
45     return r_squared
46
47 r2_score = calculate_r_squared(Y.flatten(), Y_pred.flatten())
48
49
50 print("Calculated slope (w_ols):", w_ols)
51 print(f"R-squared: {r2_score:.4f}")
52
53
```

```
54 plt.figure(figsize=(8, 6))
55 plt.scatter(x, Y, color='blue', label='Actual Data (with noise)', alpha
    =0.5)
56 plt.plot(x, Y_pred, color='red', linewidth=2, label=f'OLS Fit (R^2={
    r2_score:.4f})')
57 plt.title('OLS Fit')
58 plt.xlabel('Time squared (t^2)')
59 plt.ylabel('Distance (y)')
60 plt.legend()
61 plt.grid(True)
62
63 plt.show()
```

Further below code detailing OLS using skiti learn is presented:

```
1 #-----
2 # OLS SKLEARN
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.metrics import r2_score
8 from sklearn.linear_model import LinearRegression
9 from sklearn.model_selection import train_test_split
10
11 g = -9.81
12 sigma = 1
13 N = 100
14
15 t = np.linspace(0, 10, 100)
16 noise = np.random.normal(0, sigma, N)
17 x = t*t
18 X = x.reshape(-1, 1) #Use .reshape(-1, 1) to turn the flat list into a 2
    D Matrix column
19
20
21 def y(g, x, noise):
22     return (0.5 * g * x) + noise
23 Y = (0.5 * g * x + noise).reshape(-1, 1)
24
25
26 modelo = LinearRegression()
27
28
29 X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
    random_state=42)
30 modelo.fit(X_train, y_train)
31
32 Y_pred_test = modelo.predict(X_test)
33 # Evaluamos la calidad del modelo con los datos de TEST
34 Y_pred_plot = modelo.predict(X)
35 # predecimos sobre toda la columna X original.
36
37 w_OLS = modelo.coef_[0] #guardamos la pendiente
38
```

```
39 # la funci n oficial para R-cuadrado:
40 R2 = r2_score(y_test, Y_pred_test)
41
42 print("Calculated slope (w_OLS):", w_OLS)
43 print(f"R-squared en el set de TEST: {R2:.4f}")
44
45 plt.figure(figsize=(8, 6))
46 plt.scatter(x, Y, color='blue', label='Datos reales (Ruido)', alpha=0.5)
47 plt.plot(x, Y_pred_plot, color='red', linewidth=2, label=f'OLS (R = {R2:.4f})')
48
49 plt.title('OLS Regression')
50 plt.xlabel('t^2')
51 plt.ylabel('Distancia')
52 plt.legend()
53 plt.grid(True)
54 plt.show()
```

5.2. Ridge Regression

The following code corresponds to OLS crafted by hand, OLS using skiti learn will be presented after.

```
1 #-----
2 # Ridge
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7
8
9 g = -9.81
10 sigma = 1
11 N = 100
12  $\lambda$  = 1
13
14 I = np.identity(1)
15
16 t = np.linspace(0, 10, 100)
17 noise = np.random.normal(0, sigma, N)
18 x = t*t
19 X = x.reshape(-1, 1) #Use .reshape(-1, 1) to turn the flat list into a 2
    D Matrix column
20
21 def y(g, x, noise):
22     return (0.5 * g * x) + noise
23
24 y_calculada = y(g, x, noise)
25
26 Y = y_calculada.reshape(-1, 1)
27
28 w = np.linalg.inv(X.T @ X +  $\lambda$  * I)
29 w_ridge = w @ X.T @ Y # We use '@' for matrix multiplicatio
30 # Multiply inputs (X) by the trained slope (w_ols) to get 100 predicted
    points
31 Y_pred = X @ w_ridge
```

```
32
33 def calculate_r_squared(y_true, y_pred):
34     mean_y_true = sum(y_true) / len(y_true)
35
36     ss_tot = 0
37     for y in y_true:
38         ss_tot += (y - mean_y_true) ** 2
39
40     ss_res = 0
41     for actual, predicted in zip(y_true, y_pred):
42         # zip() takes two or more lists and pairs up their elements
43         # based on their position
44         ss_res += (actual - predicted) ** 2
45
46     if ss_tot == 0:
47         return 0.0
48
49     r_squared = 1 - (ss_res / ss_tot)
50     return r_squared
51
52 r2_score = calculate_r_squared(Y.flatten(), Y_pred.flatten())
53
54 print("Calculated slope (w_ridge):", w_ridge)
55 print(f"R-squared: {r2_score:.4f}")
56
57
58 plt.figure(figsize=(8, 6))
59 plt.scatter(x, Y, color='blue', label='Actual Data (with noise)', alpha
60            =0.5)
61 plt.plot(x, Y_pred, color='red', linewidth=2, label=f'Ridge Fit (R^2={
62            r2_score:.4f})')
63 plt.title('Ridge Fit')
64 plt.xlabel('Time squared (t^2)')
65 plt.ylabel('Distance (y)')
66 plt.legend()
67 plt.grid(True)
68 plt.show()
```

Further below code detailing Ridge using skiti learn is presented:

```
1 #-----
2 # Ridge SKLEARN
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.metrics import r2_score
8 from sklearn.linear_model import Ridge
9 from sklearn.model_selection import train_test_split
10
11
12 g = -9.81
13 sigma = 100
```

```

14 N = 100
15 alpha = 10
16
17 t = np.linspace(0, 10, 100)
18 noise = np.random.normal(0, sigma, N)
19 x = t*t
20 X = x.reshape(-1, 1) #Use .reshape(-1, 1) to turn the flat list into a 2
    D Matrix column
21
22 def y(g, x, noise):
23     return (0.5 * g * x) + noise
24 Y = (0.5 * g * x + noise).reshape(-1, 1)
25
26
27 modelo = Ridge(alpha)
28
29
30 X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
    random_state=42)
31 modelo.fit(X_train, y_train)
32
33 Y_pred_test = modelo.predict(X_test)
34 # Evaluamos la calidad del modelo con los datos de TEST
35 Y_pred_plot = modelo.predict(X)
36 # predecimos sobre toda la columna X original.
37
38 w_Ridge = modelo.coef_[0] #guardamos la pendiente
39
40 # la funci n oficial para R-cuadrado:
41 R2 = r2_score(y_test, Y_pred_test)
42
43 print(f"Pendiente (w) encontrada por Ridge: {w_Ridge:.4f}")
44 print(f"R-squared en el set de TEST: {R2:.4f}")
45
46 plt.figure(figsize=(8, 6))
47 plt.scatter(x, Y, color='blue', label='Datos reales (Ruido)', alpha=0.5)
48 plt.plot(x, Y_pred_plot, color='red', linewidth=2, label=f'Ridge (R^2={
    R2:.4f})')
49
50 plt.title(f'Ridge Regression ( $\lambda$ =alpha)')
51 plt.xlabel('t^2')
52 plt.ylabel('Distancia')
53 plt.legend()
54 plt.grid(True)
55 plt.show()

```

5.3. LASSO

```

1 #-----
2 # LASSO
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt
7 from sklearn.metrics import r2_score

```

```

8 from sklearn.linear_model import Lasso
9 from sklearn.model_selection import train_test_split
10
11
12 g = -9.81
13 sigma = 20
14 N = 100
15 alpha = 10
16
17 t = np.linspace(0, 10, N)
18 noise = np.random.normal(0, sigma, N)
19
20 x = t**2
21 X = x.reshape(-1, 1)
22 Y = (0.5 * g * x + noise).reshape(-1, 1)
23
24 modelo = Lasso(alpha)
25
26 X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
27 random_state=42)
28
29 modelo.fit(X_train, y_train)
30
31 Y_pred_test = modelo.predict(X_test)
32 # Evaluamos la calidad del modelo con los datos de TEST
33 Y_pred_plot = modelo.predict(X)
34 # predecimos sobre toda la columna X original.
35
36 w_LASSO = modelo.coef_[0] #guardamos la pendiente
37
38 # la funci n oficial para R-cuadrado:
39 R2 = r2_score(y_test, Y_pred_test)
40
41 print(f"Pendiente (w) encontrada por LASSO: {w_LASSO:.4f}")
42 print(f"R-squared en el set de TEST: {R2:.4f}")
43
44 plt.figure(figsize=(8, 6))
45 plt.scatter(x, Y, color='blue', label='Datos reales (Ruido)', alpha=0.5)
46 plt.plot(x, Y_pred_plot, color='red', linewidth=2, label=f'LASSO (R = {
47 R2:.4f})')
48
49 plt.title(f'LASSO Regression ( $\lambda$ =alpha)')
50 plt.xlabel('t^2')
51 plt.ylabel('Distancia')
52 plt.legend()
53 plt.grid(True)
54 plt.show()

```

5.4. Model Comparison

```

1 #-----
2 # MIX
3 #-----
4
5 import numpy as np
6 import matplotlib.pyplot as plt

```

```
7 from sklearn.metrics import r2_score, mean_squared_error
8 from sklearn.model_selection import train_test_split
9 from sklearn import linear_model
10
11 g = -9.81
12 sigma = 10
13 N = 100
14 alpha = 20
15
16 t = np.linspace(0, 10, 100)
17 noise = np.random.normal(0, sigma, N)
18 x = t*t
19 X = x.reshape(-1, 1) #Use .reshape(-1, 1) to turn the flat list into a 2
    D Matrix column
20
21
22 def y(g, x, noise):
23     return (0.5 * g * x) + noise
24 Y = (0.5 * g * x + noise).reshape(-1, 1)
25
26
27 OLS = linear_model.LinearRegression()
28 Ridge = linear_model.Ridge(alpha)
29 LASSO = linear_model.Lasso(alpha)
30
31
32 X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,
    random_state=42)
33 OLS.fit(X_train, y_train)
34 Ridge.fit(X_train, y_train)
35 LASSO.fit(X_train, y_train)
36
37 Y_OLS_test = OLS.predict(X_test)
38 Y_Ridge_test = Ridge.predict(X_test)
39 Y_LASSO_test = LASSO.predict(X_test)
40 # Evaluamos la calidad del modelo con los datos de TEST
41 Y_OLS_plot = OLS.predict(X)
42 Y_Ridge_plot = Ridge.predict(X)
43 Y_LASSO_plot = LASSO.predict(X)
44 # predecimos sobre toda la columna X original.
45
46 w_OLS = OLS.coef_.flatten()[0]
47 w_Ridge = Ridge.coef_.flatten()[0]
48 w_LASSO = LASSO.coef_.flatten()[0]
49
50 # la funci n oficial para R-cuadrado:
51 R2_OLS = r2_score(y_test, Y_OLS_test)
52 R2_Ridge = r2_score(y_test, Y_Ridge_test)
53 R2_LASSO = r2_score(y_test, Y_LASSO_test)
54
55 mse_ols = mean_squared_error(y_test, Y_OLS_test)
56 mse_ridge = mean_squared_error(y_test, Y_Ridge_test)
57 mse_lasso = mean_squared_error(y_test, Y_LASSO_test)
58
59 print("Calculated slope (OLS):", w_OLS)
60 print("Calculated slope (Ridge):", w_Ridge)
61 print("Calculated slope (LASSO):", w_LASSO)
```

```
62
63 print(f"R-squared en el set de TEST para OLS -> R : {R2_OLS:.4f} | MSE:
    {mse_ols:.2f}")
64 print(f"R-squared en el set de TEST para Ridge -> R : {R2_Ridge:.4f} |
    MSE: {mse_ridge:.2f}")
65 print(f"R-squared en el set de TEST para LASSO -> R : {R2_LASSO:.4f} |
    MSE: {mse_lasso:.2f}")
66
67
68 # -----
69 # COMBINED PLOT
70 # -----
71 plt.figure(figsize=(10, 6))
72
73 plt.scatter(x, Y, color='royalblue', label='Datos reales (Ruido)', alpha
    =0.4)
74
75 # Plot each history array with a unique color and label
76 plt.plot(x, Y_OLS_plot, color='red', linewidth=2, label=f'OLS (R ={
    R2_OLS:.4f})')
77 plt.plot(x, Y_Ridge_plot, color='orange', linewidth=2, label=f'Ridge (
    R ={R2_Ridge:.4f})')
78 plt.plot(x, Y_LASSO_plot, color='green', linewidth=2, label=f'LASSO (R
    ={R2_LASSO:.4f})')
79
80 plt.title(f'Comparison: OLS vs Ridge vs LASSO (lambda={alpha})')
81 plt.xlabel('t^2 (Time squared)')
82 plt.ylabel('Distancia (y)')
83 plt.legend()
84 plt.grid(True, linestyle='--', alpha=0.6)
85
86 plt.show()
```

6. AI USAGE

AI was implemented in this memoir not only to enhance the text and the register used but also as a helper in learning how to use skiti learn (alongside Datacamp's intro to skiti).